SQL Set Operations

UNION · INTERSECT · EXCEPT

A study guide using the HR schema

June 2026

What are Set Operations?

Set operations let you combine the results of two SELECT queries into one list. Instead of joining tables side by side, set operations stack results on top of each other.

There are three of them:

- UNION — combine two lists into one (no duplicates)
- INTERSECT — show only what appears in both lists
- EXCEPT — show what is in the first list but not the second

> **Note:** *All three require both SELECT statements to have the same number of columns, with matching data types.*

The Rules

Before using any set operation, these three rules must be satisfied:

- Same number of columns in both SELECT statements.
- Columns must have compatible data types — you cannot mix text and numbers.
- Column names in the result come from the first SELECT. The second one is ignored.
- ORDER BY goes at the very end, once, after the full query.

If you break any of these rules, SQL will return an error.

1. UNION

What it does

UNION combines rows from two queries into one list and removes any duplicates. If the same row shows up in both queries, you only see it once.

Think of it like merging two class registers — if a student appears in both, you still only list them once.

Syntax

```
SELECT column1, column2
FROM   first_table
UNION
SELECT column1, column2
FROM   second_table;
```

UNION vs UNION ALL

UNION ALL does the same thing but keeps every row, including duplicates. It is also faster because SQL does not have to check for duplicates.

	UNION	UNION ALL
Duplicates	Removed	Kept

Speed	Slower	Faster
Use when	You want unique rows	You want everything

Example

Get a combined list of job IDs from department 10 and department 20, with no duplicates.

```
SELECT JOB_ID
FROM   EMPLOYEES
WHERE  DEPARTMENT_ID = 10
UNION
SELECT JOB_ID
FROM   EMPLOYEES
WHERE  DEPARTMENT_ID = 20;
```

Result:

JOB_ID
AD_ASST
MK_MAN
MK_REP
IT_PROG

> **Note:** *If IT_PROG existed in both departments, UNION shows it once. UNION ALL would show it twice.*

2. INTERSECT

What it does

INTERSECT returns only the rows that appear in both queries. It finds the overlap between two results.

Think of it like finding students who are on both the football team list and the chess club list — you only want the names that appear on both.

Syntax

```
SELECT column1, column2
FROM   first_table
INTERSECT
SELECT column1, column2
FROM   second_table;
```

Things to remember

- Only rows that exist in BOTH queries are returned.
- Duplicates are removed automatically.
- The result will always have fewer or equal rows than either individual query.

Example

Find employees who are in department 80 AND earn more than 10,000 — using INTERSECT.

```
SELECT EMPLOYEE_ID, FIRST_NAME, LAST_NAME
FROM   EMPLOYEES
WHERE  DEPARTMENT_ID = 80
INTERSECT
SELECT EMPLOYEE_ID, FIRST_NAME, LAST_NAME
FROM   EMPLOYEES
WHERE  SALARY > 10000;
```

Result — only employees who meet BOTH conditions:

EMPLOYEE_ID	FIRST_NAME	LAST_NAME
145	John	Russell
146	Karen	Partners
147	Alberto	Errazuriz

> **Note:** *You could get the same result with a WHERE clause using AND. INTERSECT becomes most useful when the two queries come from completely different tables.*

3. EXCEPT (MINUS in Oracle)

What it does

EXCEPT returns rows from the first query that do NOT appear in the second query. It subtracts one list from another.

Think of it like taking the full staff list and removing everyone who showed up to a meeting — what is left are the people who did not attend.

> **Note:** *Oracle uses MINUS instead of EXCEPT. They do the same thing. If you are writing for Oracle, write MINUS. For PostgreSQL or SQL Server, write EXCEPT.*

Syntax

```
-- Standard SQL
SELECT column1, column2
FROM   first_table
EXCEPT
SELECT column1, column2
FROM   second_table;

-- Oracle
SELECT column1, column2
```

```
FROM   first_table
MINUS
SELECT column1, column2
FROM   second_table;
```

Things to remember

- Order matters — A EXCEPT B gives different results to B EXCEPT A.
- Only rows from the first query are ever returned.
- Duplicates are removed automatically.

Example

Find all departments that have NO employees in them.

```
-- All departments
SELECT DEPARTMENT_ID
FROM   DEPARTMENTS
MINUS
-- Departments that have at least one employee
SELECT DEPARTMENT_ID
FROM   EMPLOYEES
WHERE  DEPARTMENT_ID IS NOT NULL;
```

Result — departments with no employees:

DEPARTMENT_ID
120
130
140
150
160

```
170
```

> **Note:** *This is one of the most common real uses of EXCEPT/MINUS — finding records in one table that have no match in another.*

Summary

Here is a quick reminder of when to use each one:

Operation	Returns	Use it when…
UNION	All rows from both, no duplicates	You want to merge two lists
UNION ALL	All rows from both, with duplicates	You want everything, including repeats
INTERSECT	Only rows that appear in both	You want to find the overlap
EXCEPT/MINUS	Rows in A that are not in B	You want to find what is missing

Common mistakes

- Different number of columns in the two SELECT statements.
- Putting ORDER BY inside one of the SELECT statements instead of at the very end.
- Using EXCEPT when you meant to use EXCEPT the other way around — remember order matters.
- Writing EXCEPT in Oracle — Oracle needs MINUS.